02c char und Zeichensätze

02c char und Zeichensätze

00 Übersicht · Zurück: 02b Zweierkomplement · Weiter: 02d String und boolean

Der Datentyp `char`

- **Primitiver Typ** (engl. *character*), repräsentiert **einzelne** Textzeichen (Buchstaben oder Sonderzeichen).
- Literale stehen in **einfachen Anführungszeichen**.

```
char c = 'a';   // der Buchstabe a
char c = '5';   // das Zeichen 5
int i = 5;      // die Zahl 5
char c = '%';   // das Prozent-Zeichen
char c = ' ';   // das Leerzeichen
```

- Jedes Zeichen wird intern auf eine **positive Ganzzahl** abgebildet: `'a'` 97, `'5'` 53.
- Abbildung auf **16 Bit unsigned**.
- Die Abbildung ist über einen Zeichensatz standardisiert: **Unicode**.

Anmerkung zum Skript
Auf der Folie steht „0 bis 65.536“. Bei 16 Bit sind es genau $2^{16} = 65.536$ Werte, also **0 bis 65.535**.

Entwicklung von Zeichensätzen

Zeichensatz	Jahr	Eigenschaften
ASCII	1963	7 Bit, 128 Zeichen, die ersten 32 nicht druckbar
EBCDIC	1964	8 Bit, 256 Zeichen, eigenständiges Encoding, v.a. im IBM-Umfeld (Mainframes)
ISO 8859-1 bis -16	1985	8 Bit, je 256 Zeichen; die ersten 128 = ASCII. Normenreihe für alle europäischen Sprachen

- **ISO 8859-1**: u.a. deutsche Umlaute.
- **ISO 8859-15**: ISO 8859-1, unterstützt zusätzlich das **€-Zeichen**.
- Verwendung: ältere Dateien, Datenbanken, Anwendungen (z.B. ältere, meist statische Webseiten).

Themen

HM

- Der Datentyp char
- Der Datentyp String: String-Literale
- Wahrheitswerte: der Datentyp boolean
- Arithmetische Operatoren
- Container-Datentyp Array

9

Der Datentyp String

HM

- Der Datentyp "String" dient zur Darstellung von beliebig langen Zeichenfolgen.
- Zwei Varianten:
  - String-Literale
  - *String-Klasse: kommt später*
- Initialisierung von String-Literalen:

```
String word = "Haus"
String text = "Geh ins Haus!"
```

- Verkettung von String-Literalen mit + Operator (Konkatenation):

```
String compiler = "Java-" + "Compiler";
```

- Verkettung von String-Literalen mit anderen Datentypen möglich (Typkonversion in String):

```
int version = 25;
String text = "Wir arbeiten mit Java Version " + version;
```


10

Unicode und UTF

Unicode (1991, ursprünglich 16 Bit 65.536 Zeichen):
- Erste 128 Zeichen = ASCII, erste 256 Zeichen = ISO 8859-1.
- **Unicode 2.0** (seit 1996): erweitert auf **1.114.112 Zeichen**, Abbildung über **Surrogate Pairs**.
- Unterstützt alle gebräuchlichen Schriftsysteme und Zeichen weltweit.
- Jedes Zeichen hat einen festen Zahlenwert (**Codepunkt**).

UTF-x (*Unicode Transformation Format*) = Verfahren, um Codepunkte im Computer zu verarbeiten (speichern, übertragen, …):

Format	Abbildung der Codepunkte auf
UTF-8	je nach Bedarf **1 bis 4 Byte** (De-facto-Standard)
UTF-16	je nach Bedarf **2 oder 4 Byte**
UTF-32	immer **4 Byte**

> **Java**
> Java nutzt **intern** (`char` und `String`) **UTF-16**, zur Kommunikation nach außen (Dateien, Netzwerk) **UTF-8**.

Begriffe: Zeichensatz, Codepunkt, Encoding

Begriff	Bedeutung
Zeichensatz (Charset)	Menge typischerweise zusammengehöriger Zeichen
Codemenge	Menge numerischer Werte (Codepunkte)
Codepage	Zuordnungstabelle (Zeichen Zahl)
Encoding	Festlegung der **Speicherdarstellung** (Bytes)

Beispiel Unicode: Zeichensatz → *Codepage* → Codemenge (Codepunkte) → *Encoding UTF-8 bis 32* → Bytes.

Implizite Typkonversion int und char

Weil Zeichen eindeutigen Zahlen entsprechen, ist teilweise **implizite Konversion** zwischen `char` und `int` möglich:

```
int i = 'i';     // i = 105
int a = 'a';     // a = 97
int v = '5';     // v = 53
char c = 97;     // c = 'a'
int j = c;       // j = 97
char x = -2;     // Fehler (negativ, nicht im Wertebereich von char)
```

Übung dazu: 99 Übungsaufgaben